

MINISTRY OF EDUCATION, CULTURE AND RESEARCH

OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

Cuibari Vladislav FAF-243

Report

Laboratory Work No. 4

Operating Systems

Topic: The xv6 Shell from the Outside

Checked by:

Reițman Patricia, university assistant

Chișinău – 2026

CONTENTS

INTRODUCTION

The user interface of xv6 is a single program, user/sh.c, of roughly 500 lines. In this laboratory work that shell is used next to bash, so that it becomes visible, by experiment, what a shell has to do and what bash adds on top of it. A small program that prints the arguments it receives is then used to find out who expands * and who removes quotes. Before that, the sleep program from Laboratory Work No. 2 is checked against the current xv6, where the system call sleep() was renamed to pause().

The purpose of the work is to make sleep build on the current xv6, to compare the xv6 shell with bash command by command, to show with the program args which component turns *.c into file names and strips the quotes, and to read the matching code in user/sh.c.

The report follows the four parts of the handout; the first number of every figure is the number of the part. The experiments were done on Ubuntu 26.04.1 LTS running in an Oracle VirtualBox virtual machine, under the user account dalvgame. The bash side of the comparison was run in an empty folder ~/shtest that contains one file, README. The xv6 side was run at the $ prompt of xv6-riscv started with make qemu in ~/xv6-riscv; the tools were installed and checked with the course script os2026-setup.sh (QEMU 10.2, compiler riscv64-linux-gnu-gcc). The deliverables sleep.c, args.c, args_linux.c and the Makefile are stored in the lab4/ folder of the course repository together with this report.

1 FIXING SLEEP FOR THE CURRENT XV6

This chapter corresponds to Part 1 of the handout. In August 2025 the xv6 authors renamed the system call sleep() to pause(), so the sleep program of Laboratory Work No. 2 may no longer build.

1.1 Checking the Name Used by the Tree

cd ~/xv6-riscv

grep -n "pause\|sleep" user/user.h

25:int pause(int);

The evidence for this exercise is shown in Figure 1.1.



Figure 1.1 – The grep result in user.h

Comment and observation

The grep found only one declaration, on line 25 of user/user.h: int pause(int);. There is no declaration of sleep, so this tree uses the new name and the call in user/sleep.c has to be pause.

1.2 The Updated sleep.c

```
#include "kernel/types.h"

#include "user/user.h"

int

main(int argc, char *argv[])

{

if(argc != 2){

fprintf(2, "usage: sleep <ticks>\n");

exit(1);

}

pause(atoi(argv[1])); // was sleep() before Aug 2025
```

exit(0);

}

The evidence for this exercise is shown in Figure 1.2.

```
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$
```

Figure 1.2 – The updated sleep.c

Comment and observation

The program requires exactly one argument; otherwise it writes usage: sleep <ticks> to descriptor 2 (standard error) and exits with status 1. The argument is converted with atoi, the process is paused for that many clock ticks, and it exits with status 0. The Makefile contains the line $U/_sleep\ in the UPROGS list (it starts with a TAB, ends with a backslash and has nothing after it), so mkfs copies the program into the xv6 file system.

1.3 Build, Boot and Test

make qemu

$ sleep 20

$ sleep

usage: sleep <ticks>

The evidence for this exercise is shown in Figure 1.3.

```
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-
.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
$
```

Figure 1.3 – sleep running in xv6, with and without an argument

Comment and observation

The build passed with -Wall -Werror. In xv6, sleep 20 waits twenty ticks; sleep without an argument prints usage. QEMU exits with Ctrl-A, then X.

2 BASH VERSUS THE XV6 SHELL

This chapter corresponds to Part 2 of the handout. Every command was run twice: in bash, in the empty folder ~/shtest that contains a file README, and at the xv6 $ prompt.

2.1 The Commands Run in bash

mkdir ~/shtest && cd ~/shtest && touch README

echo hello > f ; cat < f

ls | grep READ

(echo a; echo b) | wc

echo one ; echo two

echo bg &

ls *

echo $HOME

echo "a b"

mkdir d ; cd d ; ls .. ; cd ..

Output:

hello

README

2 2 4

one

two

[1] 7059

bg

README f

[1]+ Done echo bg

/home/dalvgame

a b

README d f

The evidence for this exercise is shown in Figure 2.1.

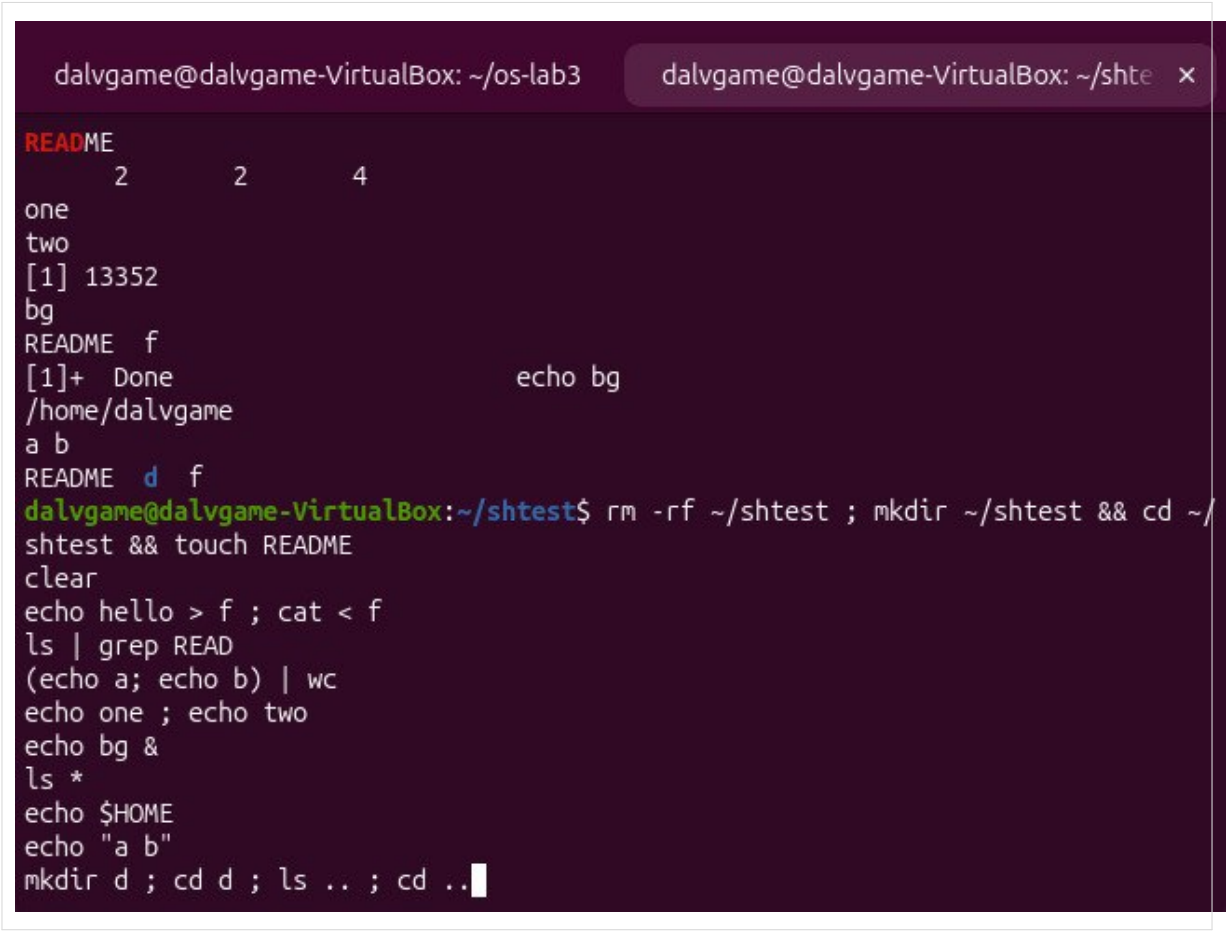


Figure 2.1 – The commands run in bash

Comment and observation

Everything worked in bash. hello was written to f and read back; the pipe passed the listing through grep; the group (echo a; echo b) produced two lines which wc counted as 2 2 4; ; ran the two echo commands one after the other; & started echo bg in the background (job [1], later reported as Done); ls * listed README and f; $HOME became /home/dalvgame; the quotes of "a b" were removed; and after cd d the command ls .. still found the program and listed the parent folder.

2.2 The Same Commands in xv6

$ echo hello > f ; cat < f

hello

$ ls | grep READ

README 2 2 2441

$ (echo a; echo b) | wc

2 2 4

$ echo one ; echo two

one

two

$ echo bg &

bg

$ ls *

ls: cannot open *

$ echo $HOME

$HOME

$ echo "a b"

"a b"

$ mkdir d

$ cd d

$ ls ..

exec ls failed

The evidence for this exercise is shown in Figure 2.2.



Figure 2.2 – The same commands run in xv6

Comment and observation

Redirection, the pipe, the group, the sequence and the background command work in the xv6 shell just as in bash. Globbing and variables do not: ls * looks for a file whose name is the single character *, and echo $HOME prints the text $HOME. Quotes are not removed, so echo "a b" prints the quote characters. The ls of xv6 prints one line per file (name, type, inode, size) and not a bare name, which is why ls | grep READ shows README 2 2 2441. After cd d the command ls .. fails (see 2.3).

One more difference was found by accident, when a Linux command line was typed at the xv6 prompt: mkdir -p ~/shtest && cd ~/shtest && touch README was answered with leftovers: cd ~/shtest && touch

README and syntax. The xv6 shell has no && (and no ||) and no ~; the parser stopped at the unknown operator and reported a syntax error.

2.3 ls Failing after cd d, and the Alternative

$ mkdir d

$ cd d

$ ls ..

exec ls failed

$ /ls ..

(the listing of the root directory)

$ /ls .

. 1 27 32

.. 1 1 1024

The evidence for this exercise is shown in Figure 2.3.

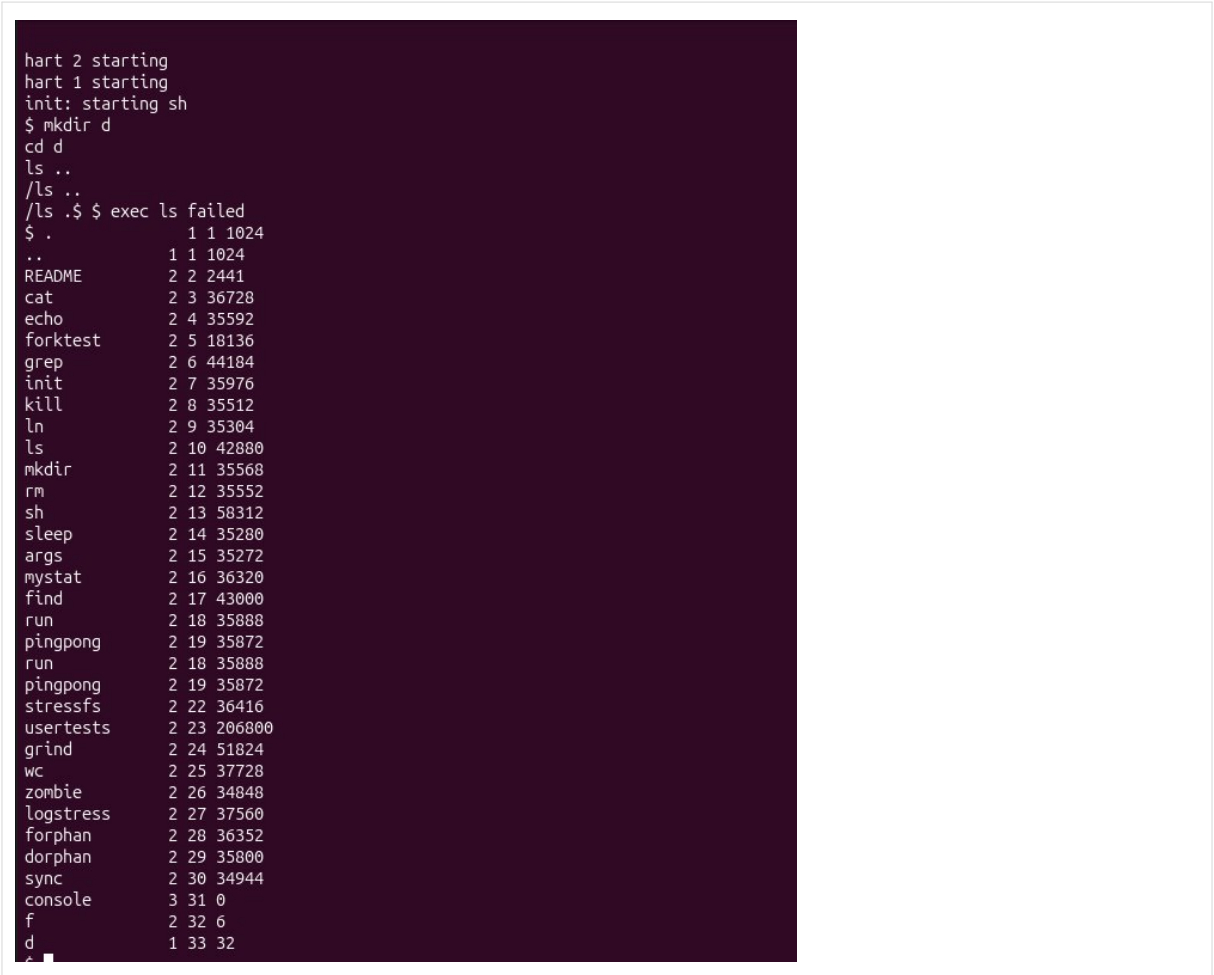


Figure 2.3 – ls failing after cd d, and the working alternative

Comment and observation

After cd d the shell is inside the folder d, and ls .. fails with exec ls failed. The xv6 shell has no PATH: it passes the bare name ls to exec, which looks for a file of that name in the current directory, and all xv6 programs are stored in the root directory /. Giving the full path, /ls .., finds the program from any directory

and prints the listing of the root (it also shows the new folder d and the file f created earlier). /ls . shows the contents of the current folder d, which holds only . and ...

2.4 The Comparison Table

Table 2.1 – bash and the xv6 shell, command by command

Command	Tests	bash	xv6
echo hello > f ; cat < f	redirection	hello	hello – the shell itself opens the files for < and >
ls | grep READ	pipe	README	README 2 2 2441 – the shell joins two processes with a pipe; the xv6 ls prints name, type, inode, size
(echo a; echo b) | wc	grouping, list	2 2 4	2 2 4 – parentheses and ; are parsed by the xv6 shell
echo one ; echo two	sequence	one, two	one, two
echo bg &	background	[1] 7059, bg, later Done	bg – the command runs in the background, but there is no job number and no Done message (no job control)
ls *	globbing	README f	ls: cannot open * – nobody expands *
echo $HOME	variables	/home/dalvgame	$HOME – the shell has no variables
echo "a b"	quotes	a b	"a b" – quotes are passed to the program as ordinary characters
mkdir d ; cd d ; ls ..	PATH	README d f	exec ls failed – no PATH, programs are looked up in the current directory only
arrow up, Tab, Ctrl+R	history, completion	work (line editing library)	do not work – the keys are echoed as text such as ^[[A; confirm with your own test

2.5 Answers to the Observe Questions

(1) Features of both shells and features of bash only. Both shells can run a program with arguments, redirect input and output (<, >), join programs with pipes (|), run lists of commands (;), group commands with parentheses, start commands in the background (&) and change the directory with the built-in cd. Only bash has globbing (*, ?, [ ]), variables and their expansion, quote removal, a PATH search, job control (jobs, the

Done message), && and ||, command history, Tab completion with line editing, and scripting constructs such as if, for and functions.

(2) Why does ls stop working after cd d? The xv6 shell has no PATH. It calls exec("ls", ...), and exec opens a file with that name relative to the current directory. The programs of xv6 live in the root directory, so as soon as the current directory is d, there is no file ls there and exec fails. The way around is to give the path of the program: /ls .. (or ../ls ..) runs it from inside d, as shown in Figure 2.3.

(3) Which missing feature would I miss most, and which is easiest to add? In daily work I would miss history with Tab completion most (together with globbing): every long command and every file name would have to be typed in full and any mistake retyped. The easiest feature to add is a PATH search: when exec fails, the shell would simply try the same name in a fixed list of directories such as /, which is only a few lines in runcmd. History and completion are much harder, because the shell would have to read single keystrokes and handle the terminal escape sequences instead of reading whole lines.

3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

This chapter corresponds to Part 3 of the handout. The program args prints the number of arguments it received and every argument inside square brackets, so nothing that happens to the command line before the program starts can stay hidden.

3.1 The Program args.c for xv6

```
#include "kernel/types.h"

#include "user/user.h"

int

main(int argc, char *argv[])

{

printf("argc = %d\n", argc);

for(int i = 0; i < argc; i++)

printf("argv[%d] = [%s]\n", i, argv[i]);

exit(0);

}
```

The line $U/_args\ was added to UPROGS in the Makefile, right after $U/_sleep\ (lines 143–145 of the Makefile hold _sh, _sleep and _args):

```
143: $U/_sh\

144: $U/_sleep\

145: $U/_args\

146: $U/_stressfs\
```

The evidence for this exercise is shown in Figure 3.1.

```
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$ cd ~/xv6-riscv
cat user/args.c
grep -n '_sleep\|_args' Makefile
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
144:    $U/_sleep\
145:    $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

Comment and observation

The program uses only the xv6 headers kernel/types.h and user/user.h. argc and argv are prepared by the exec system call from the list of strings that the shell passes to it; the program itself never sees the original command line.

3.2 args in xv6

$ args one two three

argc = 4

argv[0] = [args]

argv[1] = [one]

argv[2] = [two]

argv[3] = [three]

$ args *.c "x y"

argc = 4

argv[0] = [args]

argv[1] = [*.c]

argv[2] = ["x]

argv[3] = [y"]

$ args $HOME

argc = 2

argv[0] = [args]

argv[1] = [$HOME]

$ args 'single quoted'

argc = 3

argv[0] = [args]

argv[1] = ['single]

argv[2] = [quoted']

The evidence for this exercise is shown in Figure 3.2.



Figure 3.2 – args output in xv6

Comment and observation

In xv6 the words reach the program exactly as they were typed: *.c stays *.c, $HOME stays $HOME, and the quotes are kept as ordinary characters, so "x y" is cut into the two arguments "x and y", and 'single quoted' into 'single and quoted'. A test with args * "x y" gave the same picture: the first argument was the single character *.

3.3 args_linux in bash

gcc -o args args_linux.c

./args one two three

./args *.c "x y"

./args $HOME

./args 'single quoted'

Output (the folder ~/lab4 contained args.c, args_linux.c and sleep.c):

argc = 4

argv[0] = [./args]

argv[1] = [one]

argv[2] = [two]

argv[3] = [three]

argc = 5

argv[0] = [./args]

argv[1] = [args.c]

argv[2] = [args_linux.c]

argv[3] = [sleep.c]

argv[4] = [x y]

argc = 2

argv[0] = [./args]

argv[1] = [/home/dalvgame]

argc = 2

argv[0] = [./args]

argv[1] = [single quoted]

The evidence for this exercise is shown in Figure 3.3.

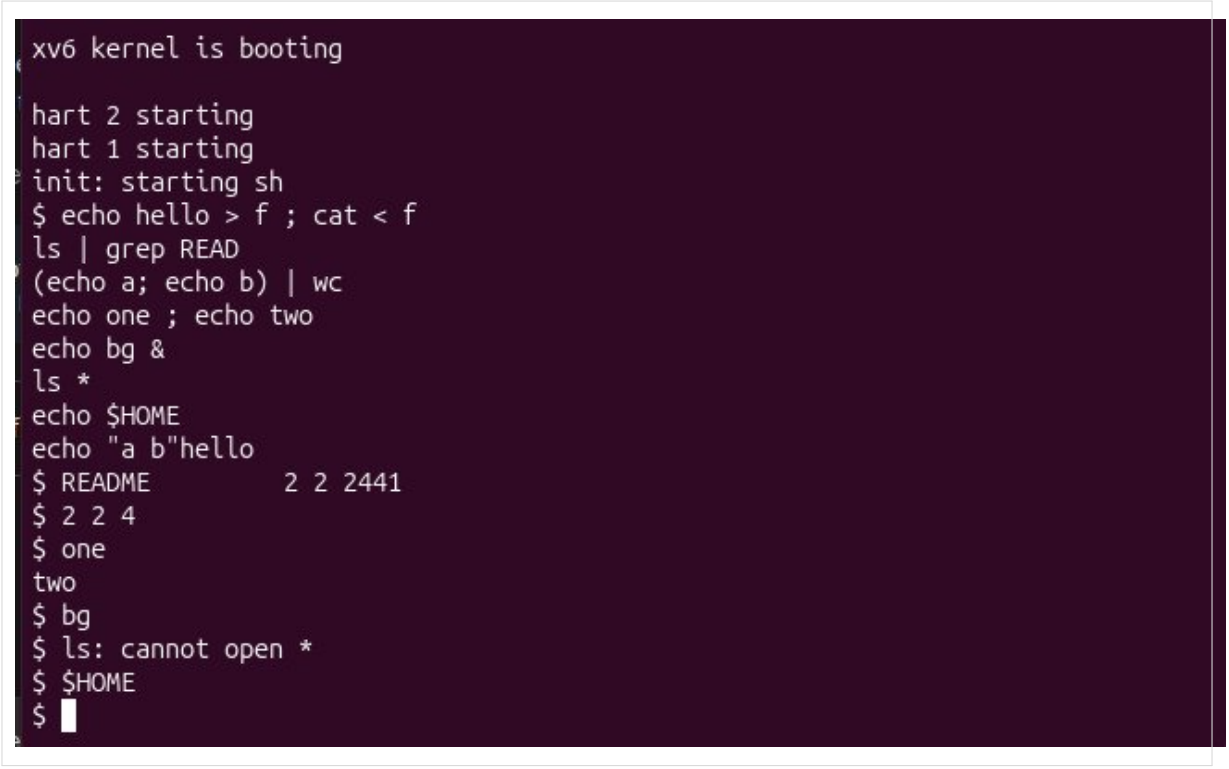


Figure 3.3 – args_linux output in bash

Comment and observation

The Linux version differs from the xv6 one only in the header (<stdio.h>) and in return 0. For ./args *.c "x y" it received 5 arguments: the program name ./args, the three file names in alphabetical order, and the single argument x y with the quotes removed and the space kept. $HOME arrived as /home/dalvgame, and 'single quoted' as one argument.

3.4 Answers to the Observe Questions

(1) Outputs for args *.c "x y".** In bash (Figure 3.3) the program received 5 arguments./args, args.c, args_linux.c, sleep.c and x y. In xv6 (Figure 3.2) it received 4: args, *.c, "x and y". The difference is exactly the work that bash does before starting the program: expanding the pattern and removing the quotes.

(2) Who turns *.c into file names and removes the quotes?** Neither the program nor the kernel: the shell does it. bash expands the pattern and strips the quotes first and only then callexec; the kernel merely copies

the finished argument strings into the new process, and the program reads them from argv. The xv6 shell does not do either of these steps, which is why the program there gets the raw text.

(3) What happened to the extra spaces in one two three? They disappeared. The shell cuts the line into words at white space and passes only the words, so any number of spaces between two words is just a separator. Both args one two three and args one two three give the same argv, and this also holds in xv6 (the tokenizer skips white-space characters, see Section 4.4).

4 READING THE SHELL

This chapter corresponds to Part 4 of the handout. In ~/xv6-riscv the commands of the handout were run and the parts of user/sh.c they point to were read.

4.1 The Kinds of Commands

wc -l user/sh.c

499 user/sh.c

grep -n "#define" user/sh.c

8:#define EXEC 1

9:#define REDIR 2

10:#define PIPE 3

11:#define LIST 4

12:#define BACK 5

14:#define MAXARGS 10

The evidence for this exercise is shown in Figure 4.1.

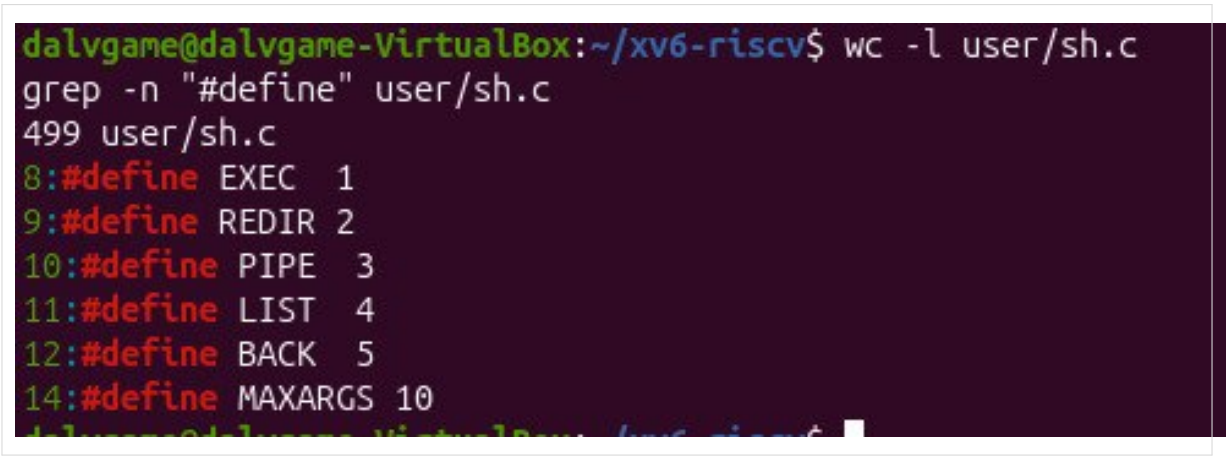


Figure 4.1 – The command kinds defined in sh.c

Comment and observation

The whole shell is 499 lines long. The #define lines name the five kinds of command the shell knows: EXEC, REDIR, PIPE, LIST and BACK. A sixth constant, MAXARGS, limits a simple command to 10 arguments.

4.2 Where Each Kind Is Run

grep -n "case " user/sh.c

75: case EXEC:

83: case REDIR:

93: case LIST:

101: case PIPE:

125: case BACK:

284: case 0:

286: case '|': ... (lines 287-294: ( ) ; & < >)

391: case '<':

394: case '>':

397: case '+': // >>

469: case EXEC: ... (lines 475-493: REDIR PIPE LIST BACK)

The evidence for this exercise is shown in Figure 4.2.



Figure 4.2 – Where each kind is run

Comment and observation

The case labels fall into three groups. In runcmd (lines 75 to 125) there is one branch per kind: run a program, open the redirection file, run both sides of a list, create a pipe and two children, run a command in the background. In gettoken (lines 284 to 294) the characters |, (,), ;, &, < and > are recognised as symbols, and in parseredirs (lines 391 to 397) the cases <, > and + handle the redirection operators, + being the second character of >>. Finally nulterminate (lines 469 to 493) walks the finished tree once more, again with one branch per kind.

4.3 The Code That Handles cd

grep -n "cd" user/sh.c | head

170: fprintf(2, "cannot cd %s\n", cmd + 3);

The evidence for this exercise is shown in Figure 4.3.

```
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$ grep -n "cd" user/sh.c | head
sed -n 155,180p user/sh.c
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
      break;
    }
  }

  // Read and run input commands.
  while (getcmd(buf, sizeof(buf)) >= 0) {
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
      cmd++;
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
  }
  exit(0);
}

void
```

Figure 4.3 – The code that handles cd

Comment and observation

Only one line of sh.c contains the text cd: the error message cannot cd %s on line 170, which belongs to the loop in main that reads a line with getcmd. Just above it (lines 161 to 164) the loop skips leading blanks and tests for an empty command; a line that starts with cd is then handled right there, with a call to chdir on the text after the third character (cmd + 3), and the loop goes on to the next line without any fork. Only the other lines are executed in a child process.

4.4 Answers to the Observe Questions

(1) The five kinds of command. EXEC – a program with its arguments (a plain word); REDIR – input or output redirection, produced by < and > (and >>); PIPE – two commands joined by |; LIST – commands run one after the other, produced by ;; BACK – a command run in the background, produced by &. Parentheses only group a command and have no kind of their own.

(2) Where is cd handled and why is it not a separate program? In main, in the loop that reads a line, before the shell forks. The current directory is a property of a process. A separate program would run in a child process, change the directory of the child and exit, and the shell would still be in the old directory. Only the shell process itself can change its

own directory, exactly as with the built-in cd of bash in Laboratory Work No. 3.

(3) Code that handles * or $.** There is none. The search fo* and $ returned only lines in which * is the C pointer syntax (char *argv[], struct cmd *cmd, casts such as (struct execcmd *)cmd), and the only $ is the prompt string "$ " written by getcmd on line 137. The tokenizer knows only white space and the symbols |, (,), ;, &, <, >; everything else is a word that is passed on unchanged. This explains Part 2: there is no globbing and no variables, so ls * looks for a file called * and echo $HOME prints $HOME. The same tokenizer treats a quote as an ordinary character, which is why args showed "x and y" as two arguments, and it skips white space between words (lines 278 and 303), which explains why extra spaces disappear.

CONCLUSION

The sleep program was adapted to the current xv6 by replacing the call sleep() with pause(), the name declared on line 25 of user/user.h; it builds with -Wall -Werror and works with and without an argument.

The side-by-side comparison showed that a shell of 499 lines already provides the core of a Unix shell: programs with arguments, redirection, pipes, lists, grouping, background execution and a built-in cd. Everything that makes bash comfortable (globbing, variables, quote removal, && and ||, a PATH search, job control, history and completion) is added on top. The missing PATH explained why ls fails after cd d and why /ls .. works.

The program args made the division of work visible: in bash the shell turns *.c into file names and removes the quotes before exec, and the program receives the result (5 arguments), while in xv6 the program receives the text as typed (4 arguments). Reading user/sh.c confirmed it: the five kinds of command are the whole language, cd is handled by the shell itself because the working directory belongs to the shell process, and no code handles * or $.

REFERENCES

[1] Operating Systems, Laboratory Work 4 handout. Lab 4 – The xv6 Shell from the Outside (Track B). FCIM / FAF / UTM, 2026–2027.

[2] MIT PDOS. xv6-riscv source repository. https://github.com/mit-pdos/xv6-riscv.

[3] R. Cox, F. Kaashoek, R. Morris. xv6: a simple, Unix-like teaching operating system. MIT 6.1810. https://pdos.csail.mit.edu/6.1810/.

[4] Free Software Foundation. GNU Bash Reference Manual. https://www.gnu.org/software/bash/manual/.